

Requirements, Design and Test Documentation

Dining Philosophers für VSP

Hert, Tom

Tom.Hert@haw-hamburg.de

Zacharias, Laurin

Laurin.Zacharias@haw-hamburg.de

2. Dezember 2024

Inhaltsverzeichnis

1	Aufgabenstellung	3
2	Anforderungen	4
2.1	Offene Fragen	4
3	Entwurf	5
4	Remote Prochure Calls	6

1 Aufgabenstellung

Sie setzen eine Lösung des bekannten Philosophenproblems um. Einzelne Teilnehmer, hier Philosophen, werden in einzelnen Container realisiert. Diese kommunizieren ausschließlich mittels Nachrichten und die Zugriffe der Teilnehmer werden verteilt koordiniert, d.h. keine (pseudo-) zentrale Lösung. Es sollen Performance-Messungen für 5, 10 und eine von Ihnen zu ermittelnde maximal Anzahl von Container realisiert werden. Die RPC Lösung ist hierbei komplett selbst zu erstellen und durch Tests abzusichern, bestehende Lösungen wie `grpc`, `rmi`, ... sind nicht zugelassen.

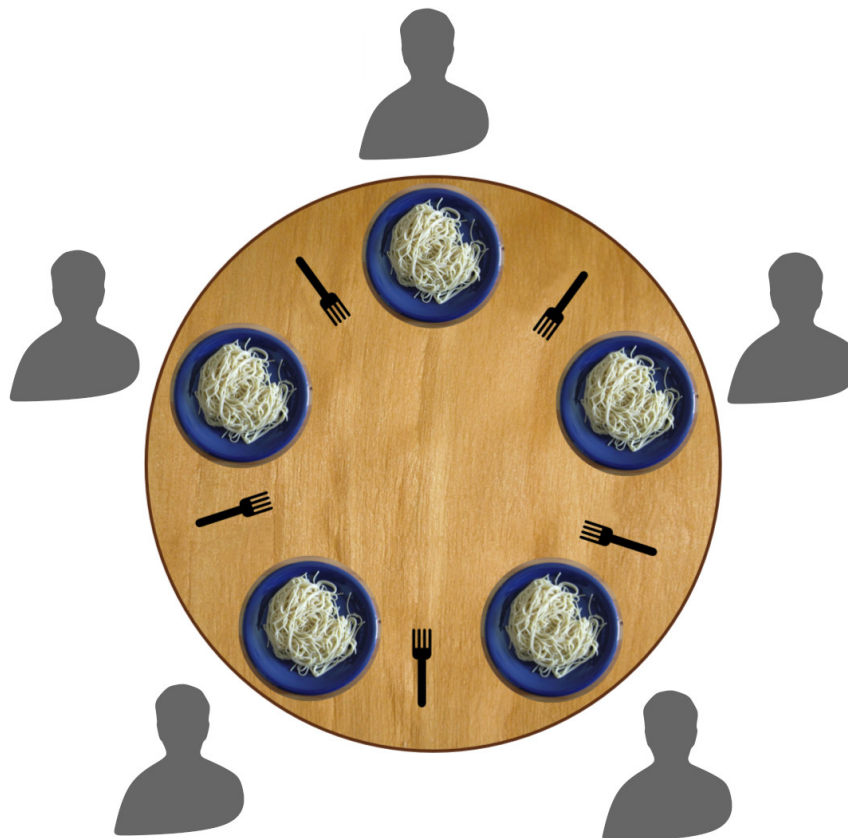


Abbildung 1: Dining Philosophers

2 Anforderungen

1. Besteck kann nur von einem Philosophen gleichzeitig benutzt werden.
2. Es gibt genauso viele Bestecke wie Philosophen.
3. Philosophen müssen einzelne Container sein.
4. Philosophen werden können entweder Essen oder Denken:
 - (a) wenn sie Denken interagieren sie nicht mit anderen Philosophen oder Besteck.
 - (b) wenn sie Essen wollen benötigen sie die 2 angrenzenden Bestecke um diese Tätigkeit zu vollführen und legen diese anschließend wieder ab.
5. Philosophen sollen ausschließlich die 2 angrenzenden Bestecke benutzen können.
6. Philosophen können nicht wissen wann die anderen Philosophen Essen oder Denken wollen.
7. Philosophen müssen im Wechsel Essen und Denken können sodass sie nicht verhungern.
8. Philosophen verhungern wenn sie nicht Essen können.
9. Philosophen dürfen ausschließlich per Nachrichten kommunizieren und ihre Zugriffe sollen verteilt koordiniert werden.
10. Die Performancemessungen sollen mit 5, 10 sowie einer zu ermittelnden maximal Anzahl stattfinden.
11. Zur verteilten Koordination soll eine eigene Lösung für RPC genutzt werden und keine Bibliothek.
12. Die eigene RPC Lösung muss mit Tests abgesichert werden.

2.1 Offene Fragen

- wie sollen die Zeitspannen in den Zuständen aussehen: feste Zeitspannen, zufällig? - Soll verhungern eine Zeitgrenze haben oder erst wenn ein deadlock entsteht?

3 Entwurf

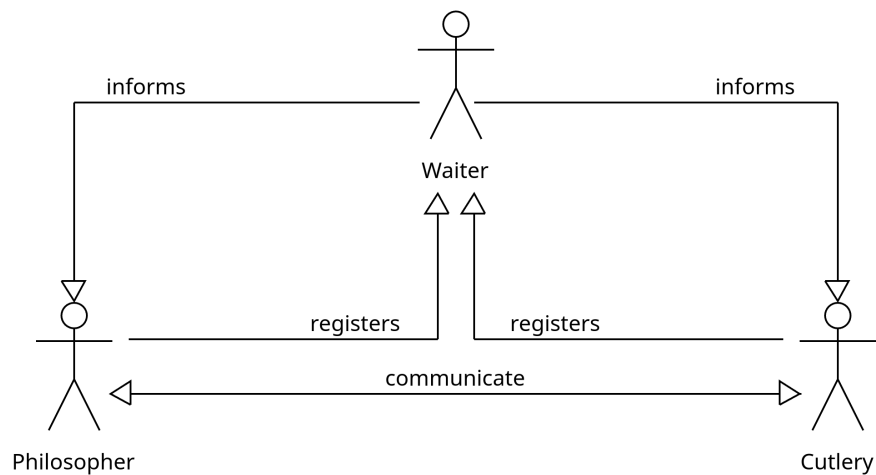


Abbildung 2: Das Setup der Container

Zur Kommunikation zwischen den verschiedenen Containern nutzen wir ein zentrales Melderegister namens Waiter. In diesem müssen sich Cutlery und Philosopher registrieren, um miteinander kommunizieren zu können. Der Waiter ist ein zentraler Bestandteil des Systems und wird von allen Containern benötigt, um sich gegenseitig zu finden. Die Kommunikation zwischen den Containern erfolgt über das Publish-Subscribe-Prinzip, d.h. dass man nach der Registrierung beim Waiter über jeden anderen registrierten Container informiert wird.

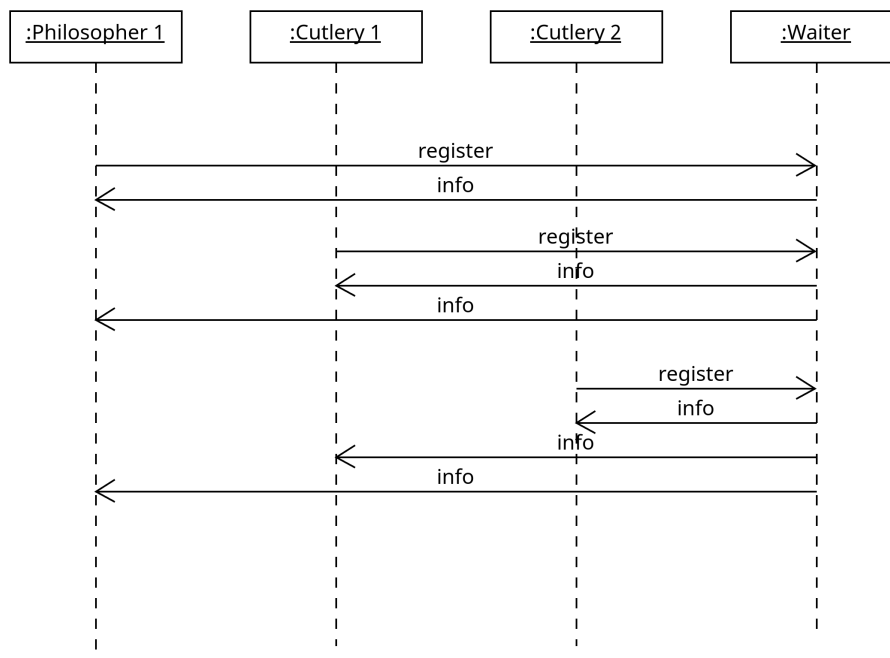


Abbildung 3: Das Publish Subscribe Prinzip vom Waiter

4 Remote Prodeure Calls

Die RPCs sind mit einem Trait 'Calls' zur Bestimmung der Signaturen aller remote rufbaren Funktionen der Container deklariert. Zur Kommunikation nutzen wir TCP Sockets über die wie folgend Nachrichten zum Funktionsaufruf als bytes codiert gesendet werden:

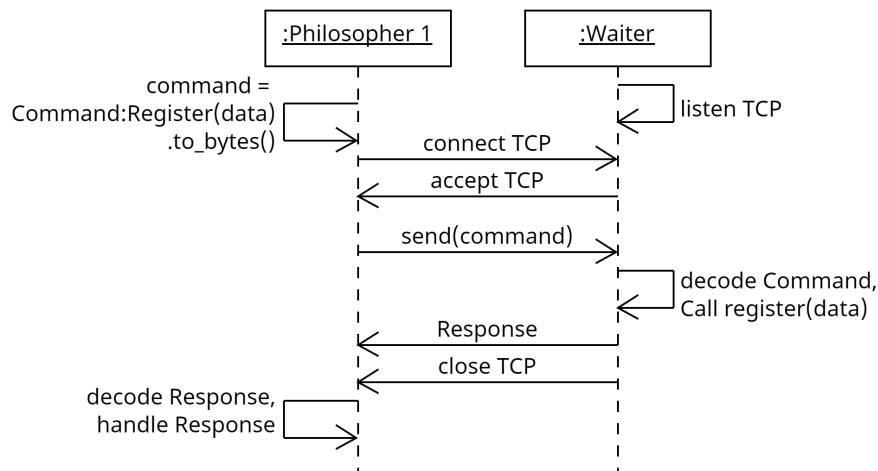


Abbildung 4: RPC Ablauf

- **Commands** ist ein enum das alle möglichen Funktionsaufrufe an Nodes umfasst und beim Senden eines RPCs in bytes codiert und gesendet wird und anschließend vom Empfänger ins enum decodiert und als Funktionsaufruf ausgeführt wird.
- Das Antwortformat auf einen RPC ist ein enum **Response**: (Success, Failure(String), Return(Vec<u8>), NotFound), das je nach Ausgang des Aufrufs Daten zurückliefern kann.

Abstrahiert sieht das ganze für einen Philosophen oder Waiter dann so aus:

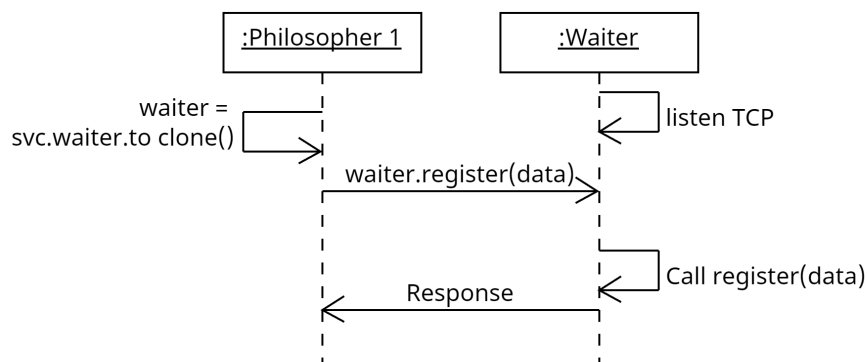


Abbildung 5: RPC Abstrahiert